HOCHSCHULE KONSTANZ
TECHNIK, WIRTSCHAFT UND GESTALTUNG

System Architecture

Elevator control and Human Machine Interface

Vlad Stan

Maxim Petrovici

Pius Pfleghar

Minh Triet Do

Prof. Dr. Alexander Krupp

SS 25

Table of contents

1 Overview 1
1.1 List of group participants 1
1.2 Project Plan 1
1.3 Responsibilities 1
1.4 Design 2
1.5 Description of the communication interface HMI 2
1.6 Access Policy 2
1.7 Diagram 4
2 HMI 5
2.1 Design 5
2.2 System Overview 5
2.3 Architecture 5
2.4 Access Control 5
2.5 Communication 6
2.6 GUI Behavior 6
2.7 Logging 6
3 Implementation HMI 7
3.1 System Structure 7
3.2 Classes 7
3.2.1 Access Control 7
3.2.2 Outside Control 8
3.2.3 Inside Control 8
3.2.4 Admin Control 8
3.3 HMI Logic 8
3.4 Communication with OPC UA 9
3.5 Logging 9
4 Evaluation HMI 10
4.1 Functional Testing 10
4.2 Access Control Behavior 10
4.3 System Responsiveness 10
4.4 Fault Tolerance and Error Handling 10

4.5 Transparency / Logging 10
5 Control API 11
5.1 System overview 11
5.2 Elevator state automaton 11
5.3 Elevator commands 11
5.4 OPC-UA server 11
5.5 Interfacing between the system's elements 11
6 Implementation Control API 12
6.1 Java control program 12
6.1.1 Main class 12
6.1.2 ElevatorStateAutomaton class 12
6.1.3 CommandLineInterface class 12
6.1.4 MessageProcessing class 12
6.1.5 Reciever class 12
6.1.6 Sender class 12
6.1.7 ElevatorData class 13
6.2 Python OPCUA server program 13
7 Evaluation 14
7.1 HMI 14
7.2 Control API 14
7.2.1 Elevator state automaton routine testing 14
7.2.2 OPC-UA server testing and information transfer testing 14
7.2.3 Responsiveness 14
7.2.4 Error handling 15
8 Scalability & Potential Improvements 16

List of illustrations

Figure 1: HMI-ServerControl Communication 4
Figure 2: Software Architecture 5
Figure 3: Login UI 7
Figure 4: Outside Control UI 8
Figure 5: Inside Control UI 8
Figure 6: Admin Control UI 8
Figure 7: Log 9

Declaration of Independent Work

We hereby declare that the present work was completed independently and without outside assistance, and that no resources other than those indicated were used.
In particular, we affirm that all direct and indirect quotations from other sources have been properly acknowledged.

Konstanz, July 5th, 2025

Pius Pfleghar, 307024

Minh Triet Do, 307408

Vlad Stand, 314622

Maxim Petravici, 314621

1 Overview

1.1 List of group participants

Pius Pfleghar	HMI
Minh Triet Do	HMI
Vlad Stand	Control API
Maxim Petravici	Control API

1.2 Project Plan

Task	Project Domain	Estimated Hours
1	OPC UA communication (Client)	25 h / Pius
2	HMI structure setup	18 h / Triet
3	Implement main UI views: level buttons, elevator status panel, door control	20 h / Triet
4	Integrate logging features (stops, alarms, interactions)	25 h / Pius
5	Implement Access Control	14 h / Triet
6	Full integration and testing with control system, final documentation and system validation	18 h / Pius & Triet
7	OPC UA communication (Server)	20 h / Max
8	Message management and interpretation	17 h / Vlad
9	PLC, Control system communication	17 h / Vlad
10	Control algorithm design and implementation	26 h / Max & Vlad
11	Control system testing and documentation	20 / Max

1.3 Responsibilities

Logging & OPC UA Communication with the Control API.....**Pius Pfleghar**
Access Control & HMI and Graphical UI.....**Triet Do Minh**
Control algorithm for controlling the elevator.....**Maxim Petrovici & Vlad Paul Stan**
Communication with the HMI API.....**Maxim Petrovici**
Communication with the elevator simulation.....**Vlad Paul Stan**

1.4 Design

Component Name	Description
HMI	Human-Machine Interface for interacting with the elevator (UI, logging).
OPC UA Client (HMI)	Manages communication between HMI and Control via OPC UA protocol.
Control System Server	Manages elevator logic and exposes OPC UA variables for control/monitoring.
Logging Module	Captures and stores user actions, elevator events, and system alarms.
Access Control	Handles user roles and access levels (Operator/Supervisor).

1.5 Description of the communication interface HMI

The Human-Machine Interface (HMI) communicates with the elevator's control system using the OPC UA (Open Platform Communications Unified Architecture) protocol. OPC UA is a platform-independent, service-oriented communication standard widely used in industrial automation for secure and structured data exchange between clients and servers.

In this project, the HMI functions as the OPC UA client, while the elevator control system acts as the OPC UA server. The HMI sends control commands such as target floor selection or door operations by writing to specific OPC UA variables (nodes) on the server. At the same time, it continuously monitors the elevator's state, such as position, motor status, and door status, by subscribing to relevant data nodes. This enables real-time visualization and user feedback within the HMI.

OPC UA ensures robust and scalable communication. It supports hierarchical data models, event-based updates, and a publish-subscribe mechanism, allowing the HMI to respond immediately to changes in the control system. The protocol also enables future expansion, such as integrating user authentication and encrypted messaging, though these are not implemented in the current project phase.

1.6 Access Policy

To ensure modularity and clear separation of responsibilities, each OPC UA variable is designated with a one-way access policy. Writable variables are controlled exclusively by the HMI, allowing user inputs to be transmitted to the control system without risking unintended feedback loops. Conversely, readable variables are updated solely by the control system and observed by the HMI to reflect the current system status.

For example, targetLevel and directionRequest are writable from the HMI side, while elevatorPosition, doorStatus, and motorState are strictly read-only for the HMI. This structured access policy maintains a clean interface, prevents data conflicts, and enables safe monitoring and control within the system.

Variable Name	Type	Direction	Description
targetLevel	Int	HMI → Control	Target level selected from the cabin
directionRequest	Boolean	HMI → Control	Up (True) / Down (False) direction request HMI
doorCommand	Boolean	HMI → Control	Open (True) / Close (False) door command
reset	Boolean	HMI → Control	Reset
speedSupervisor	Int	HMI -> Control	upV1 = 1, upV2 = 2, downV1 = -1, downV2 = -2
crawlSelect	Float	HMI -> Control	Speed between -5 and 5
elevatorPosition	Int	Control → HMI	Current level of the elevator
doorStatus	String	Control → HMI	"open", "closed", "opening", "closing"
userAccessLevel	String	HMI Internal	"User" / "Supervisor"
log	String	HMI Internal	Stores log messages (arrival, stop, alarm)
controlAPIlog	String	Control → HMI	Contains the logs from the Control API

Direction	Type	Purpose
HMI → Control	OPC UA Write-only	User commands (target level, direction, door)
Control → HMI	OPC UA Read-only	Elevator state (position, door, motor)
HMI (internal)	Logging (Text)	Interaction, alarm, and event logging

1.7 Diagram

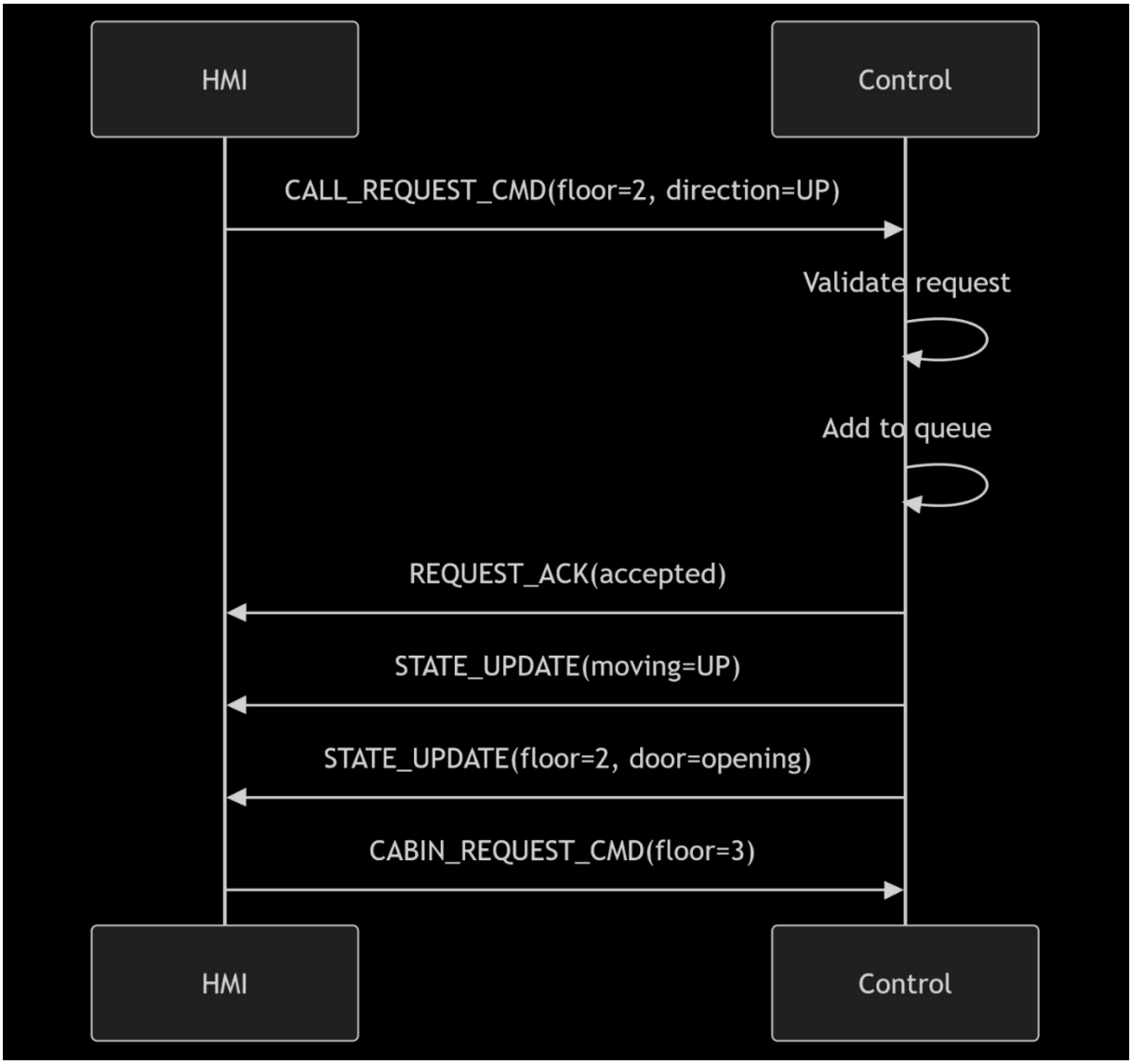


Figure 1: HMI-ServerControl Communication

2 HMI

2.1 Design

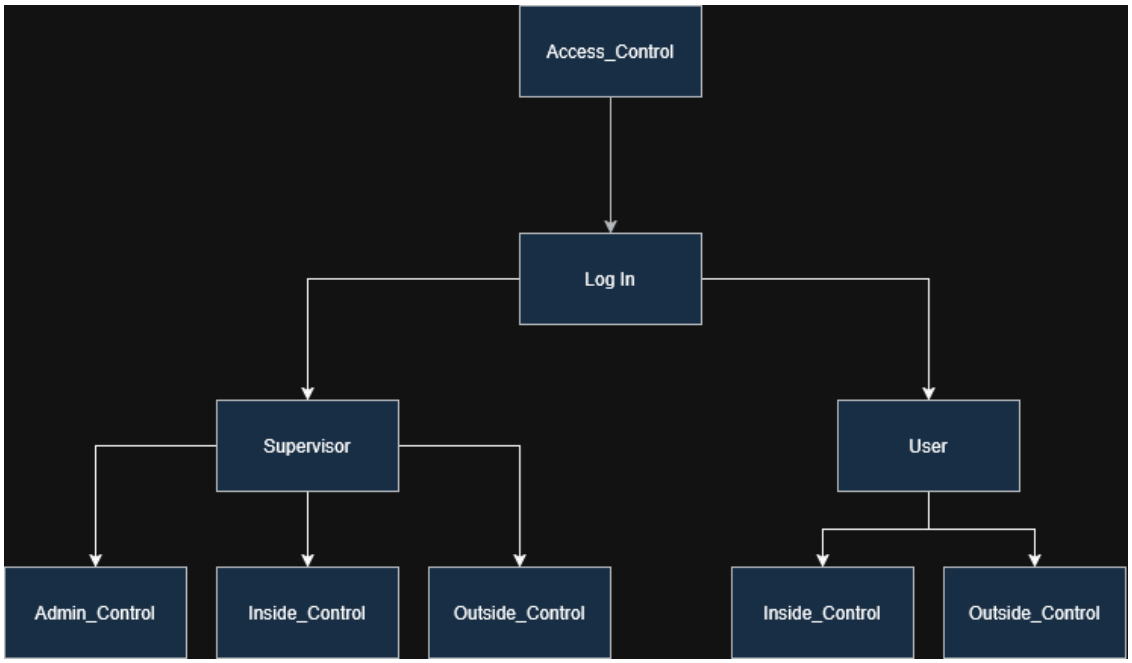
The elevator system is split into client and server sides. On the server side, the elevator logic and motion simulation are handled. On the client side, we implemented the HMI, user interaction, and access control. Communication between both sides is managed using the OPC UA protocol.

2.2 System Overview

The HMI and the OPC UA connection is built using Python and Tkinter. It provides an interface for outside (call buttons on each floor), inside (floor selection inside the elevator) and the admin control panels. Access control is applied before users can interact with the elevator. The logic for handling roles, logging events, and communicating with the server is divided into separate modules.

2.3 Architecture

The system is structured into the following Python modules:



Each component is independent and communicates via shared state or OPC UA signals.

Figure 2: Software Architecture

2.4 Access Control

All access to elevator functions is routed through the Access_Control module. The Supervisor must enter a PIN to log in and the User doesn't have a password. The system supports two roles:

- **User**: Can use basic elevator functions (call elevator, choose floor).
- **Supervisor**: Has extended access to administrative controls through admin_control.py.

If authentication fails, access is denied and an error is logged.

2.5 Communication

Communication with the server-side elevator logic is implemented through the OPCUA_Client module. It connects to the OPC UA server and either reads or writes variables such as:

- Current floor
- Target floor
- Door state
- Elevator direction

This setup allows the client to operate independently of the server implementation details.

The OPC UA client connects at startup of the User or Supervisor panel and remains active throughout the session. If the session is closed or the logout button is pressed, the OPC UA client will disconnect from the server.

2.6 GUI Behavior

The system provides two interfaces:

- **Outside control**: A simple panel to request the elevator from any floor.
- **Inside control**: A floor selection panel inside the elevator.
- **Admin control**: A panel with extra function for selected personal.

Each interface is implemented as a Tkinter window. Once the user logs in, they are redirected to the appropriate panel depending on their role. GUI events (button clicks) trigger OPC UA messages and update the system state accordingly.

2.7 Logging

The system logs events such as Button interactions and errors, but also simple state changes such as “OPC UA Client connected to …”.

Logs are saved to a file and can be extended for audit or debugging purposes. Additionally the logging inputs are displayed in the Terminal, that the User can see them live.

3 Implementation HMI

The elevator system consists of several interlinked software modules, each responsible for a distinct part of the system. The logic for motion and floor handling is implemented server-side and treated as a black box in this documentation.

3.1 System Structure

The system is structured into the following modules:

- Access_Control.py: Handles user authentication and access rights.
- admin_control.py: Supervisor-only features.
- inside_control.py: Button panel inside the elevator.
- outside_control.py: Floor call panels for users.
- elevator_state.py: Keeps track of the elevator's current floor, requests, and occupancy.
- Logging.py: Logs all relevant events and user actions.
- OPCUA_Client.py: Handles communication between the HMI and the server logic via OPC UA.

The HMI is built using Tkinter and is split into multiple windows depending on user role and functionality (login, floor control, admin settings, etc.).

3.2 Classes

The system is structured into several Python classes, each handling a distinct part of the HMI logic. This modular approach keeps the responsibilities separated and the code maintainable. All GUI-related behavior is managed through Tkinter, while backend communication happens via OPC UA. Core classes include access handling, elevator controls (inside and outside), administrative functions, logging, and communication.

Each class interacts with the shared OPC UA client and, where necessary, with the elevator state polling system. The central access control ensures that user roles (User or Supervisor) are consistently checked before executing commands.

3.2.1 Access Control

Access control is a core component. Every user must log in to access the system. Users are assigned a role, either "User" or "Supervisor." Only Supervisors can access restricted features like system crawl speed and reset.

Login credentials are verified by Access_Control.py. Successful login returns user data, including role and permitted actions. Failed attempts are logged and rejected.

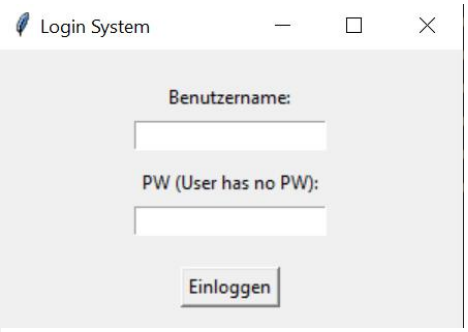


Figure 3: Login UI

3.2.2 Outside Control

The Outside_Control class implements the elevator call panels placed on each floor. Each floor (except the top and bottom) has two buttons: one for upward and one for downward requests. The buttons send a target floor along with a direction to the backend via OPC UA. Pressed buttons are highlighted in yellow until the elevator arrives.

The GUI also includes a floor status indicator using a canvas that shows the current position of the elevator. This is updated periodically by polling the OPC UA server. Each request is logged, and button highlights are reset once the elevator reaches the requested floor.

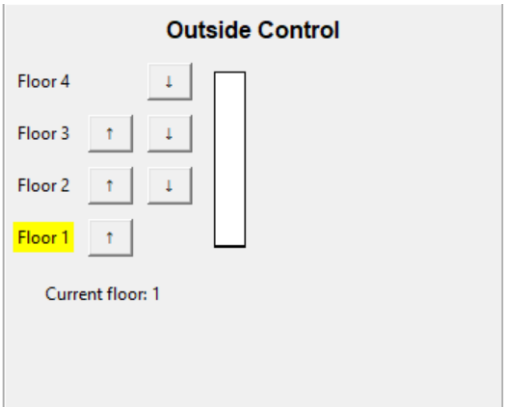


Figure 4: Outside Control UI

3.2.3 Inside Control

The Inside_Control class represents the control panel inside the elevator cabin. It provides buttons for each floor, which become active after authentication. Like the outside panel, floor selections are sent to the control server through the OPC UA client.

In addition, the panel includes door open/close controls and a status display showing the elevator's position and door state. Updates are received from the backend and reflected in real-time. Inside requests are visually confirmed by button color changes and removed once fulfilled.

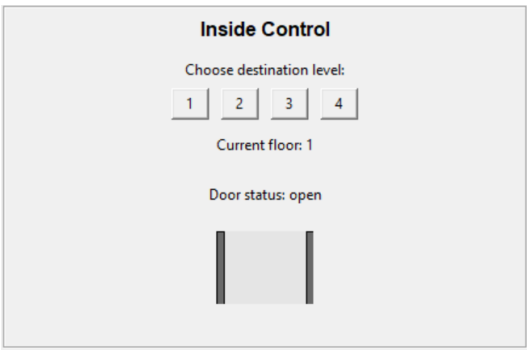


Figure 5: Inside Control UI

3.2.4 Admin Control

The Admin_Control class is exclusive to supervisor users. It includes reset functions, crawl mode activation, and a speed override. These commands are not accessible to regular users and are guarded behind a second verification step.

The admin view also includes a logging pane to display the latest control API events, making it easier to verify system actions during maintenance or debugging. Like the other views, all data exchange uses OPC UA, and actions are recorded in the centralized log.

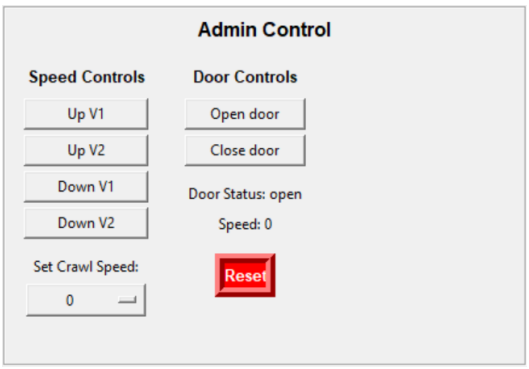


Figure 6: Admin Control UI

3.3 HMI Logic

Each button press from the inside or outside panels is processed via the HMI and triggers a request:

- **Inside**: Adds a floor request to the elevator queue.
- **Outside**: Sends a call signal from a floor to request the elevator.

Tkinter is used to dynamically update the GUI with current floor, door status, and pending requests.

The elevator state (position, direction, door) is read via OPC UA variables using the OPCUA_Client.py module. Communication is handled with periodic polling and value updates.

3.4 Communication with OPC UA

All commands and status updates are exchanged with the server-side elevator logic through OPC UA. The client reads and writes specific node values:

- Floor requests
- Current floor
- Door state
- Active user session

The OPC UA client connects at startup and remains active throughout the session. If the connection fails, the system logs the error.

3.5 Logging

All significant actions (logins, button presses, state changes) are logged with timestamps in Logging.py. This includes:

- Login attempts (success or fail)
- Floor calls (inside and outside)
- Reset
- Speed selection from the Supervisor panel
- Door control
- Connection/Disconnection from the OPC UA
- System events (startup, shutdown)

Logs are stored in a text file and can be reviewed for debugging or analysis.

The log input has the following layout:
date/time – name – level (info, warning or error) – message

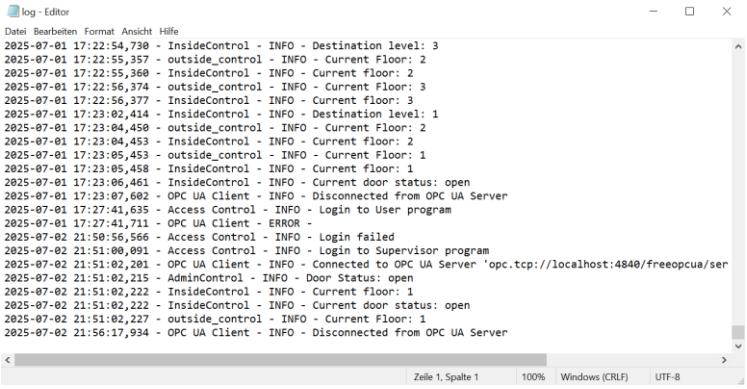


Figure 7: Log

4 Evaluation HMI

The elevator control system was tested in a simulated environment where the logic of movement and floor transitions is handled by a server-side module. The client (HMI) interacts with it via OPC UA, sending commands and receiving state updates. Our evaluation focuses on correctness, responsiveness, role-based behavior, and system robustness.

4.1 Functional Testing

The system was tested using both roles: *User* and *Supervisor*. Basic tasks like calling the elevator, selecting floors, and observing state changes (e.g., door open/close, floor indicator) worked as intended. Requests were correctly sent to the server, and feedback was processed in real time through the GUI.

4.2 Access Control Behavior

One of the key focus areas was role-based access. In tests, restricted actions like accessing the supervisor menu or overriding the elevator behavior were only permitted for users logged in with the Supervisor role. Attempts by regular users were blocked, and appropriate logs were created.

Sample log entry (from Logging.py):

```
2025-07-01 18:10:30,223 - Access Control - INFO - Login to User program
```

This showed that the access control layer effectively filters requests before execution.

4.3 System Responsiveness

GUI response times were long but stable. Floor selection buttons gave immediate visual feedback, and state indicators updated with a few second delay. Since the OPC UA client runs asynchronously, communication with the elevator controller remained smooth most of the time, with too many commands the system took some time to process them.

4.4 Fault Tolerance and Error Handling

Various edge cases were tested:

- Selecting an invalid floor
- Trying to send a command while the door was in motion
- Disconnecting the OPC UA server during runtime

In each case, the system either ignored invalid actions or returned a controlled error message. When the server connection was lost, the GUI froze updates, without crashing.

4.5 Transparency / Logging

All interactions were recorded in log files. This included button presses, successful and denied access attempts, and system state changes. Logs made it easy to trace user behavior during test runs and verify that the expected logic had triggered. This ensures an easy way to debug a system or find the correct module where an error is in. Also this way the transparency of the system is high.

5 Control API

5.1 System overview

While the client side is taken by the HMI, the server side consists of the control program. The way the control program works is by having multiple threads which communicate between each other through public static variables. We have 2 main parts in the control program. One takes care of creating the OPC-UA server and the other is controlling the elevator itself. The OPC-UA server is created with a python script that will get the control data from the HMI, then the control part is done in a Java program and using TCP communication to communicate with the OPC-UA server.

5.2 Elevator state automaton

The elevator state automaton implements the elevator control as a state automaton, in which every state implemented describes an action of the elevator. The main states implemented are the following: 1. Stationary state

2. moving up and moving down states with the fast speed
3. moving up and moving down states with the slow speed
4. moving up and moving down states with the slow speed
5. waiting for the elevator to stop

5.3 Elevator commands

The elevator commands are Strings that when called modify some static variables which are responsible with changing the states of the elevator. The main commands are the following:

1. reset - command for resetting the automaton and PLC simulation
2. close/open - opening and closing the door
3. 1/2/3/4 - setting the target floor
4. up1/up2/down1/down2 - setting the movement speed and direction
5. crawl[speed] - setting the crawl speed
6. stop - stopping the motor no matter the speed

5.4 OPC-UA server

This part implements in python the OPC-UA server. The HMI client will connect to this server and send the command data. From here the server will then send the data to the control part and receive the needed data that should be send to the HMI client.

5.5 Interfacing between the system's elements

The main way the main parts communicate is through TCP-IP port communication where we send and receive jsons with the relevant data from or to the HMI.

6 Implementation Control API

6.1 Java control program

The java program consists of 4 main threads and 2 additional classes and the Main class. The four threads are: ElevatorStateAutomaton, CommandLineInterface, MessageProcessing, Reciever. The classes are the following.

6.1.1 Main class

Here the 4 threads are declared and started. Besides that the Main class has public static variables used for communication between threads like: target, doorState, reset, curentState, speed or floor.

6.1.2 ElevatorStateAutomaton class

Here the implementation is done with some local variables describing the state of the elevator. At every iteration of the main loop, these variables are calculated or copied from the static variables. The way the state is analyzed is through a switch case. In every case there are if clauses that check for change in state and call a routine for initializing a certain state.

6.1.3 CommandLineInterface class

This has 2 implementations. One for the command line input from the user and the other is an implementation takes care of getting command from the MessageProcessing thread. The way it approaches this is by a switch case that sets the corresponding static variables which later trigger a change in state in the state automaton thread.

6.1.4 MessageProcessing class

The message processing thread takes care of interpreting the data from the HMI and giving the correct commands. The main issue is how we make the elevator respond to certain inputs from the users like multiple requests from different users at different floors. The way this is done is through a main queue where commands are stored and 2 auxiliary queues which are sorted and then depending on the current direction of movement they are introduced in the main queue. Every time a command is executed the head of the queue is removed and it goes to the next command.

6.1.5 Reciever class

This is the thread that takes care of getting the data through a TCP port from the OPC-UA server. This and the Sender class use the gson java library to extract data from json packets and get the variables which in turn are the commands from the HMI.

6.1.6 Sender class

This is an auxiliary class which is used to send data to the OPC-UA server regarding the state of the elevator like the current floor or door state. The methods are called in multiple threads depending on what is executed at a moment and what needs to be sent.

6.1.7 ElevatorData class

This is an auxilliary class that is used by the sender to standardize the variables that will be send back to the OPC-UA server.

6.2 Python OPCUA server program

The python OPC UA server contains the setup for OPC UA, defining the endpoint, namespace and main node. After that there are the variables with the same names as in the HMI project, so that the HMI can access them, and they are set to writable so that they can modify the command variables. Next there are 2 endpoints, /update, where the server receives a json from the java program and updates the corresponding variables in the server, and /values, where the java program gets a json with the command variables from the server, that are modified by the HMI. The code also contains the setup for the connection through a socket with the java program.

7 Evaluation

7.1 HMI

The HMI system developed steadily over the course of the project. Initially, the goal was to create a basic control interface for the elevator, with simple state display and floor selection. As the project progressed, the interface was expanded to include role-based access, logging, and administrative functions like reset and crawl mode.

The modular design helped keep the code manageable and made it easier to divide tasks between team members. Early in the project, we agreed on the structure of the interface, including variable names, and expected behavior, so both sides could develop independently while ensuring compatibility. This coordination made it possible to treat the server like a black box during HMI development, without losing consistency or functionality.

Several improvements were made along the way based on testing:

- Clearer error messages for login failures
- Improved formatting in the log view
- Smoother state polling and GUI updates

Overall, the project evolved in a focused and practical way, guided by testing and feedback from the other group.

7.2 Control API

7.2.1 Elevator state automaton routine testing

The testing of the state automaton was done through the use of the command line interface with testing all commands. Besides that each state was checked, and the change conditions were verified and confirmed working.

The second part of testing was seeing if the data received from the HMI is interpreted correctly. We tested several possible combinations of inputs from the HMI to see the elevator response.

7.2.2 OPC-UA server testing and information transfer testing

Here we tested if the data transfer is done correctly. For example, initially we tried receiving data from the client. After fixing and correcting any errors we checked if the control part coud send data back to the client.

7.2.3 Responsiveness

We checked if the elevator responds well to user input. The response is pretty slow unfortunately. We also tested the limits and saw that sometimes the sensors responsible for state changes do not work properly.

7.2.4 Error handling

Error handling is done for every instruction that requires it. For example reading sensors can fail throwing errors. If that happens then we will get a console message regarding what failed, and depending on the exception itself try reconnecting to the PLC simulation or retrying the execution of the instructions stopped.

8 Scalability & Potential Improvements

While the current implementation works well, there are several areas where it could be improved:

- **Alternative GUI frameworks:** Tkinter was a simple and lightweight choice, but other frameworks like PyQt or Kivy would offer more flexibility in design and responsiveness.
- **Connection feedback:** Right now, the OPC UA connection is silent in case of failure. A visible status indicator would improve user awareness.
- **Expanded user roles:** More fine-grained roles beyond User and Supervisor could help in larger systems with different levels of access.
- **UI enhancements:** Visual indicators (icons, colors) for system state and permissions would make the interface easier to understand.

These suggestions are not critical but could enhance the reliability, scalability, and usability of the system in a next phase.